Post Quantum Session Authentication for Drone Command Links: Executed Baselines, Software in the Loop and Replay Rules

Huda Mueen
Independent researcher, United Arab Emirates
hudamueen2407@gmail.com

***Abstract*—Drones and ground robots authenticate commands over narrow radio links with keys that a quantum adversary may later recover. Published work measures post quantum primitives on microcontrollers or post quantum TLS on hosts; it rarely compares complete command channel designs under one set of conditions, with the flight stack in the loop and with the replay rule that decides availability. We execute five designs end to end, from MAVLink 2 signing as shipped to a hybrid session (X25519 plus ML-KEM-768, ML-DSA-44 signatures by both sides) and a signature on every command, on two open RISC-V core designs in cycle accurate RTL simulation, and run them with real cryptography in front of ArduPilot in software in the loop over an emulated telemetry radio. The hybrid session costs 1.76 to 1.90 times the vehicle side computation of a classical session of the same shape, while a signature per command costs 292 to 371 times a per message tag. In the loop, the hybrid session keeps the share of commands acknowledged within 0.5 s, outages included, at 0.931 against 0.935 without authentication, while a signature per command drops it to 0.418. A strict next counter rule, which earlier proofs assumed, costs -0.033 of that share on paired seeds; a replay cache costs nothing measurable. ProVerif proves injective agreement on commands for a replay cache that accepts out of order delivery. The code, data, logs and scripts are public at https://github.com/hudasol/post-quantum-security.**

I. Introduction

A small multirotor runs its flight stack on a microcontroller and receives commands over a telemetry radio of tens of kilobits per second. MAVLink 2, the common protocol, offers a per message tag with a pre shared key [1]; key establishment and endpoint authentication are left to the operator, and classical public key schemes used for them are threatened by a future quantum computer. The replacements are standardised: ML-KEM for key establishment [2] and ML-DSA for signatures [3].

What an engineer cannot yet answer from published evidence is how a complete post quantum command channel compares with the classical and the shipped alternatives on the same device, the same link and the same flight stack, and what the replay rule that the protocol needs does to availability. Primitive benchmarks on Cortex-M cores [4] and post quantum TLS on embedded or host systems [5]–[7] answer parts of this; formal analyses of drone handshakes rarely include the per message layer.

This paper contributes: (1) executed baselines of five command channel designs on two open RISC-V core designs in cycle accurate RTL simulation, with correctness checks, stack and code size, under identical compiler and libraries (Section V); (2) software in the loop runs of ArduPilot with a gateway that executes the real handshakes and per message tags, three replay rules on paired seeds, and a mission model per rule (Section VI); (3) ProVerif models of the hybrid session with three design fixes and of replay rules that accept out of order delivery (Section VII); (4) a pre registered, script checked method in which every number in this paper is read from a committed result file.

II. Related work

Kannwischer et al. established the pqm4 method for Cortex-M4 measurements [4]. Post quantum TLS has been measured on embedded clients [5], and KEMTLS replaces handshake signatures with encapsulation [6], [8]; post quantum WireGuard does the same for a VPN handshake with a machine checked model [9]. Firmware signatures have been measured on low power boards [10] and in secure boot [11]. Closest to our setting, Hyoung et al. evaluate post quantum key exchange for unmanned system command channels with a TLS 1.3 handshake over an emulated UAV radio on a laptop [7], de Meneses et al. design and formally verify a post quantum EDHOC variant for UAVs with estimated costs [12], and MAVSec adds encryption to MAVLink [13]. Twardokus et al. show for vehicle to vehicle messages that frame loss, not computation, can decide whether post quantum authentication is practical [14]. Table I compares the closest of them with this paper. We differ in executing every design under one set of conditions on core designs, in putting a real flight stack in the loop, and in treating the replay rule as a design parameter that is both proved and measured.

III. Design and baselines

All session designs share four flights and the MAVLink 2 signing field of 13 B per message after the handshake [1]. *Classical*: X25519 key exchange, Ed25519 signatures by both sides over the transcript and both identities, key confirmation tags. *Post quantum only*: ML-KEM-768 and ML-DSA-44 in the same roles. *Hybrid* (proposed): X25519 and ML-KEM-768 together, the session key derived from both secrets and a transcript hash, ML-DSA-44 signatures by both sides. Our first formal pass found that a design in which only the ground station signs leaves the vehicle unauthenticated; the vehicle

TABLE I
CLOSEST PRIOR WORK AND THIS PAPER (AS DESCRIBED IN EACH SOURCE)

Work	Platform	Link and stack	Replay rule	Formal
[7]	x86 laptop	emulated UAV radio, TLS 1.3 proxy	not studied	none
[5]	ESP32, Cortex-A53, others	Ethernet or Wi-Fi, TLS	not studied	none
[8]	Xeon server	emulated network, TLS	not studied	proof on paper
[12]	estimated costs	UAV, EDHOC variant	not studied	Tamarin
[14]	V2V chipset	DSRC, simulated and on road	not applicable	proof on paper
This paper	two RISC-V core designs (RTL), host	emulated SiK radio, ArduPilot in the loop	three rules, measured and proved	ProVerif

TABLE II
BYTES PER HANDSHAKE FLIGHT AS BUILT BY THE EXECUTED PROTOCOLS

Design	F1	F2	F3	F4	Total
Classical	32	32	80	80	224
PQ only	1,184	1,088	2,436	2,436	7,144
Hybrid, F1	1,216	1,120	2,436	2,436	7,208

F1 and F3 ground to vehicle, F2 and F4 vehicle to ground; MAVLink framing and link fragmentation not included.

signature (change F1) closes it. Two further fixes concern the provisioned key fallback (fresh nonces, F2) and firmware updates (a signed version with a monotonic counter, F3). Table II gives the bytes of each flight. *Signing as shipped*: a provisioned key, no handshake. *Signature per command*: an ML-DSA-44 signature on every command, verified by the vehicle. The vehicle accepts a command under one of three rules: *published* (timestamp greater than the last accepted, gaps allowed) [1], *strict* (only the next counter; a loss stalls the stream until resynchronisation), and *replay cache* (any counter not yet accepted within a window).

IV. METHOD

RTL simulation. Ibex in its small configuration [15] and CV32E40P [16], both RV32IMC without caches or wait states, are simulated cycle accurately with Verilator. One firmware image per design runs the vehicle and the ground side, each step timed with the cycle counter, five runs per image with a fixed seed block; every run checks that both sides derive the same key, that all signatures and tags verify, and known answers for the hash functions. Libraries are pinned and unmodified: mlkem-native [17], mldsa-native [18] and Monocypher [19], compiled with one compiler at `-O3`; peak stack is measured by stack painting.

Software in the loop. ArduPilot Copter 4.7.2 [20] flies scripted missions with its built in vehicle model behind a relay that emulates a SiK class telemetry radio with frame loss, retransmission and one outage per mission. A gateway pair in front of the ground station and the flight stack executes the handshake, the tags and the replay rule with the same algorithms on the host; vehicle side compute is applied as a delay equal to the RTL cycles of the slower core at 168 MHz, so the host speed does not enter the result. Seeds are paired across configurations, so every configuration sees the same outages and command phases. Seven configurations fly 20 missions each: no authentication, signing as shipped,

TABLE III
EXECUTED BASELINES, MILLION CYCLES PER UNIT (RTL SIMULATED, MEDIAN OF FIVE RUNS)

	Vehicle		Ground	
Design	Ibex	CV32E40P	Ibex	CV32E40P
MAVLink 2 tag, 60 B[a]	14,705	13,426	14,263	13,055
Classical session	15.4	12.4	15.4	12.4
PQ only session	26.3	19.2	21.6	15.4
Hybrid session, F1	29.4	21.9	35.1	26.1
ML-DSA-44 per command	5.4	3.9	–	–

[a] cycles, not millions: check (vehicle) and generation (ground) of one tag over a 60 B frame.

the classical session, the hybrid session under each of the three rules, and ML-DSA-44 per command; acknowledged shares carry bootstrap intervals over missions, and differences between rules are paired by mission. A mission model with 1000 runs per configuration gives availability per rule for comparison.

Formal models. ProVerif [21] checks secrecy and injective agreement under a Dolev Yao attacker with abstract primitives, unbounded sessions and the bounds stated per model.

Discipline. 14 predictions with refutation clauses were hashed before the runs and scored by script under the less favourable reading of any ambiguous clause: 12 hit, 1 between and 1 missed (the miss: a seen counter table read by concurrent branches gives a counterexample, so the cache must check and record in one step). Every number in this paper is registered with its file, column and query and checked by a script in continuous integration.

V. EXECUTED BASELINES ON CORE DESIGNS

Table III and Fig. 1 give the cost of one session start and of one command. On both cores the hybrid session costs 1.76–1.90× the classical vehicle side and only 1.12–1.14× the post quantum only session: the X25519 part is cheap beside the two ML-DSA-44 operations, and one X25519 multiplication (3.16 M cycles on CV32E40P) costs more than an ML-KEM-768 encapsulation (1.70 M). The hybrid handshake carries 7,208 B against 224 B for the classical one and needs 50.5 kB of stack against 2.1 kB. After the handshake a command costs a tag check of 13,426 cycles on a 60 B frame; verifying an ML-DSA-44 signature on a COMMAND_LONG instead costs 292× the tag check of the same frame. At 168 MHz the vehicle side of the hybrid session start would take about 130 ms on CV32E40P (estimated), against 74 ms for the classical session.

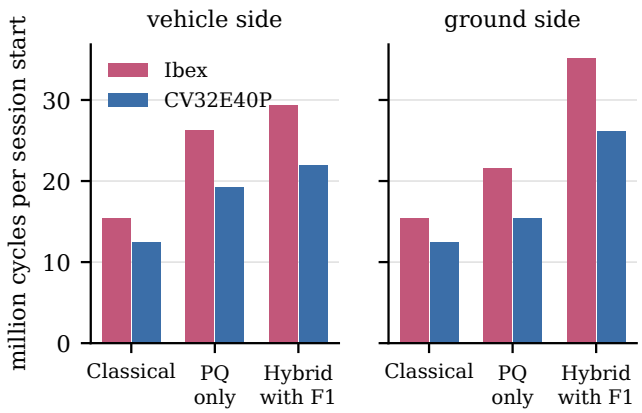


Fig. 1. Million cycles of one session start per design, vehicle and ground side (RTL simulated, median of five runs).

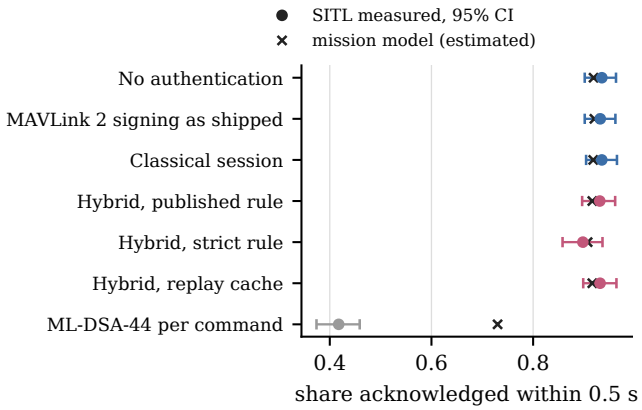


Fig. 2. Share of commands acknowledged within 0.5 s with one outage per mission, per configuration: ArduPilot in the loop (dots, 95% interval over 20 missions) and the mission model (crosses).

Ibex needs 1.34× the cycles of CV32E40P for the same hybrid handshake, so a figure for one core of an instruction set does not transfer to another.

VI. Software in the Loop and Availability

Table IV and Fig. 2 show that once a session exists, its per message cost is that of the shipped tag: the hybrid session adds 4.4 ms to the median command round trip against 4.0 ms for signing as shipped. Its handshake takes 1.49 s over the emulated radio against 0.19 s for the classical session, a cost paid once per session. A signature on every command adds 496.1 ms to each round trip, and only 0.418 of its commands are acknowledged within the deadline (mission model: 0.730), because the deadline falls inside the transfer of a 2.4 kB signature.

The replay rule matters more than the algorithms once a session exists. Link layer retransmission delivers some frames after newer ones; the published rule rejected 11 of them as late. The strict next counter rule lost 58 commands while it resynchronised and rejected 13 that arrived ahead of the expected counter, lowering the acknowledged share by -0.033 (paired 95% CI -0.047 to -0.019). The replay cache accepted the reordered frames: its difference to the published rule is +0.0006 (+0.0002 to +0.0013). The mission model, whose link layer delivers in order, puts the strict rule cost at only -0.0010 against 0.991 under the published rule; reordering, which only the flight stack in the loop exposed, is what makes the strict rule expensive. Even in the model the strict rule cost grows with the outage rate, to -0.0092 at 60 outages per hour.

VII. Formal Results

ProVerif checked 86 queries in 24 model files: 62 proved, 21 counterexamples (in the stated designs before F1 to F3, in deliberate controls and as expected findings) and 3 that the tool could not prove. With F1 the session keys of both sides stay secret and each side authenticates the other. For out of order delivery, a replay cache that checks and records each (session, counter) pair in one step proves injective agreement on commands with several commands per key, while a control without the check and an encoding in which concurrent branches read the table give counterexamples; a bounded window could only be proved on the command layer with help to the tool. Together with Section VI, this resolves the earlier choice between in order delivery and a stalled stream in favour of a replay cache.

VIII. Discussion and Limitations

The results support a session design for streamed commands: the post quantum cost is paid once per session, and the per message cost stays that of the shipped tag. A signature per command is a poor fit for streams but remains an option for rare, high value commands. The limits are explicit. Open RISC-V cores are not Cortex-M flight controllers: instruction set, multiplier, memory system and flash wait states differ, so the cycles are a baseline for comparison between designs, not a prediction for a given board. The link is an emulation of a radio, the flight stack runs on a host, and only ArduPilot was used. Proofs are symbolic and say nothing about an implementation; a real replay window needs a correct bounded table. Free memory matters as much as cycles: default PX4 builds on 7 board classes leave as little as 6,008 B of flash (host measured from linker output), which raises the estimated share of a fleet needing new hardware from 0.20 to 0.30 in a fleet model. *Threats to validity.* (i) Cycles come from two open core designs with ideal single cycle memories; Cortex-M flight controllers have flash wait states and caches. (ii) In the loop, vehicle side compute is a delay derived from those cycles at an assumed 168 MHz; the host was slower than that delay for a small share of per message tags, which we count but do not correct. (iii) The radio is an emulation with independent frame loss and one scripted outage per mission; real loss is bursty. (iv) Twenty missions per configuration give intervals of a few hundredths for the acknowledged share, so only the strict rule and per command signature effects are resolved. (v) Prior work numbers are not comparable to ours because platforms and links differ; Table I compares designs

TABLE IV
ARDUPILOT IN THE LOOP OVER AN EMULATED SIK CLASS RADIO, 20 PAIRED MISSIONS PER CONFIGURATION

Configuration	Handshake (s)	Added round trip (ms)	Share acknowledged within 0.5 s [95% CI]
No authentication	–	ref.	0.935 [0.902, 0.963]
MAVLink 2 signing as shipped	–	4.0	0.932 [0.902, 0.962]
Classical session	0.19	4.3	0.935 [0.904, 0.965]
Hybrid, published rule	1.49	4.4	0.931 [0.896, 0.961]
Hybrid, strict rule	1.50	4.3	0.898 [0.858, 0.936]
Hybrid, replay cache	1.44	4.2	0.932 [0.899, 0.964]
ML-DSA-44 per command	–	496.1	0.418 [0.374, 0.459]

Handshake: median, emulated link and executed cryptography with RTL derived delays. Added RTT: median round trip to the flight stack's acknowledgement against no authentication (SITL measured). Acknowledged share includes one outage per mission.

and methods, not figures. (vi) All reviews of this study were internal, not by independent groups. The next step is to repeat the executed baselines on Cortex-M4 and Cortex-M33 boards with registered predictions and a borrowed telemetry radio.

IX. CONCLUSION

Executed under one set of conditions, a hybrid post quantum session for drone commands costs about twice the computation of a classical session and over thirty times its handshake bytes, yet leaves the per command cost and the availability of the shipped tag almost unchanged, while a signature per command does not. A replay cache, proved safe in the symbolic model, removes the need to choose between in order delivery and a stalled stream.

REFERENCES

[1] MAVLink project, "Message Signing (Authentication)," MAVLink Developer Guide, https://mavlink.io/en/guide/message_signing.html, 2026.

[2] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203)," Federal Information Processing Standards, https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf, 2024.

[3] ——, "Module-Lattice-Based Digital Signature Standard (FIPS 204)," Federal Information Processing Standards, https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf, 2024.

[4] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4," Cryptology ePrint Archive 2019/844, https://eprint.iacr.org/2019/844.pdf, 2019.

[5] Bürstinghaus-Steinbach, Krauß, Niederhagen, and Schneider, "Post-Quantum TLS on Embedded Systems," AsiaCCS 2020; Cryptology ePrint Archive 2020/308, https://eprint.iacr.org/2020/308.pdf, 2020.

[6] Gonzalez and Wiggers, "KEMTLS vs. Post-Quantum TLS: Performance on Embedded Systems," SPACE 2022; Cryptology ePrint Archive 2022/1712, https://eprint.iacr.org/2022/1712.pdf, 2022.

[7] Y. Hyoung, D. Lim, J. Bang, D. Park, H. Kim, and H. Seo, "Post-Quantum Command Authentication for Unmanned Systems: A KpqC Feasibility Study under Real-Time and Bandwidth Constraints," Cryptology ePrint Archive 2026/2343, https://eprint.iacr.org/2026/2343.pdf, 2026.

[8] Schwabe, Stebila, and Wiggers, "Post-Quantum TLS without Handshake Signatures," ACM CCS 2020; Cryptology ePrint Archive 2020/534, https://eprint.iacr.org/2020/534.pdf, 2020.

[9] Hülsing, Ning, Schwabe, Weber, and Zimmermann, "Post-quantum WireGuard," IEEE S&P 2021; Cryptology ePrint Archive 2020/379, https://eprint.iacr.org/2020/379.pdf, 2021.

[10] Banegas, Zandberg, Baccelli, Herrmann, and Smith, "Quantum-Resistant Software Update Security on Low-power Networked Embedded Devices," Cryptology ePrint Archive 2021/781, https://eprint.iacr.org/2021/781.pdf, 2021.

[11] Kumar, Gupta, Chattopadhyay, Kasper, Krauß, and Niederhagen, "Post-Quantum Secure Boot," DATE 2020, https://past.date-conference.com/proceedings-archive/2020/pdf/0438.pdf, 2020.

[12] de Meneses, Teixeira, and Henriques, "Design, Formal Verification, and Evaluation of a Post-Quantum EDHOC Variant for UAV Communications," SBSeg 2026, https://sol.sbc.org.br/index.php/sbseg/article/download/44307/44070/, 2026.

[13] Allouch, Cheikhrouhou, Koubâa, Khalgui, and Abbes, "MAVSec: Securing the MAVLink Protocol for Ardupilot/PX4 Unmanned Aerial Systems," IWCMC 2019; arXiv 1905.00265, https://arxiv.org/pdf/1905.00265, 2019.

[14] Twardokus, Bindel, Rahbari, and McCarthy, "When Cryptography Needs a Hand: Practical Post-Quantum Authentication for V2V Communications," NDSS 2024, https://par.nsf.gov/servlets/purl/10492326, 2024.

[15] lowRISC, "Ibex RISC-V Core," GitHub repository, https://github.com/lowRISC/ibex, 2026.

[16] OpenHW Group, "CV32E40P RISC-V Core," GitHub repository, https://github.com/openhwgroup/cv32e40p, 2026.

[17] PQ Code Package, "mlkem-native," GitHub repository, https://github.com/pq-code-package/mlkem-native, 2026.

[18] ——, "mldsa-native," GitHub repository, https://github.com/pq-code-package/mldsa-native, 2026.

[19] Monocypher project, "Monocypher," GitHub repository, https://github.com/LoupVaillant/Monocypher, 2026.

[20] ArduPilot project, "ArduPilot," GitHub repository, https://github.com/ArduPilot/ardupilot, 2026.

[21] B. Blanchet, B. Smyth, V. Cheval, and M. Sylvestre, "ProVerif 2.05: Automatic Cryptographic Protocol Verifier: User Manual and Tutorial," Inria, https://bblanche.gitlabpages.inria.fr/proverif/manual.pdf, 2023.